

PHASE 6-PROJECT TESTING

Project Title: POCKETSMART AI

Your Smart Budget & Recommendation Assistant

College Name: Government Arts and Science College, Manur.

College Code: MSU142

Team ID: 6ab21524d329d50f0db6120b

Team Leader: Esakki Aruna S NM ID:55055706AAD495BADB04C8C4D3B478B8

Team Member: Utchi Makali NM ID: 0BBEDB52C642EC0B0BA4CB4B22DE9E2D

Team Member: Kombu Madathi NM ID: 86EE18ABFB36AEF83784FA01234C5948

Team Member: S Ulagammal NM ID: B59DED29E07C4F78BCF442535560F984

1. Introduction

The testing phase of PocketSmart AI is performed to verify that the developed application works according to the specified requirements. Testing helps identify functional errors, incorrect calculations, data-handling problems, interface issues, and integration problems before the system is released for use.

2. Objectives of Testing

- To verify that every major feature works according to its requirements.
- To identify and correct errors in the application.
- To verify that income and expense calculations are accurate.
- To ensure that budget limits and alerts work correctly.
- To verify that the AI recommendation module produces relevant suggestions.
- To check the reliability and usability of the application.
- To ensure that data is stored and retrieved correctly.

3. Testing Strategy

A systematic testing approach is followed in which individual modules are tested first and then tested together as an integrated system. Both valid and invalid inputs are considered. The application is tested from the user's point of view as well as at the individual module level.

4. Types of Testing

Testing Type	Purpose	Application in PocketSmart AI
Unit Testing	Tests individual components separately.	Login, calculations, budget functions, and recommendation functions.
Integration Testing	Checks communication between modules.	Frontend, backend, database, and AI module.
Functional Testing	Verifies that features meet requirements.	Expense entry, budget tracking, reports, and recommendations.
System Testing	Tests the complete application.	End-to-end application workflow.
Usability Testing	Checks ease of use and interface clarity.	Dashboard, forms, navigation, and messages.
Performance Testing	Checks response and processing behavior.	Transaction processing and dashboard loading.
Security Testing	Checks protection of user information.	Authentication and access to user records.

5. Test Cases

Test ID	Test Case	Input / Action	Expected Result	Status
TC01	User Registration	Enter valid user details	Account should be created successfully	Pass
TC02	User Login	Enter valid credentials	User should access the dashboard	Pass
TC03	Invalid Login	Enter incorrect credentials	Error message should be displayed	Pass

TC04	Add Expense	Enter a valid expense	Expense should be saved and reflected in totals	Pass
TC05	Add Income	Enter a valid income	Income should be saved and balance updated	Pass
TC06	Set Budget	Enter category and budget limit	Budget should be saved successfully	Pass
TC07	Budget Calculation	Add expenses against a budget	Remaining budget should be calculated correctly	Pass
TC08	Expense Category	Select a category for an expense	Expense should appear under the selected category	Pass
TC09	Dashboard	Open dashboard after transactions	Correct financial summary should be displayed	Pass
TC10	AI Recommendation	Provide spending data	Relevant recommendation should be generated	Pass
TC11	Transaction History	Open transaction history	Saved transactions should be displayed correctly	Pass
TC12	Invalid Input	Submit incomplete or invalid data	Validation message should be displayed	Pass

6. Functional Testing

Functional testing verifies the main operations of the application. User authentication, transaction management, budget management, dashboard calculations, reports, and

AI recommendations are checked against their expected behavior. Both normal and invalid inputs are tested to ensure proper validation.

7. Integration Testing

Integration testing verifies that the modules communicate correctly. For example, when an expense is added, the database should store the transaction, the dashboard should update the expense total, the remaining budget should be recalculated, and the recommendation module should be able to use the updated spending information.

8. Database Testing

Database testing ensures that user information, income records, expense records, categories, and budgets are stored accurately. Data insertion, retrieval, updating, and deletion operations are verified. The system is also checked to ensure that one user's financial records are not incorrectly displayed for another user.

9. AI Recommendation Testing

The AI recommendation component is tested using different spending patterns. Test data includes normal spending, high category-wise spending, and spending that approaches or exceeds budget limits. The generated recommendations are checked for relevance, consistency, and clear presentation to the user.

10. Usability Testing

Usability testing focuses on whether users can understand and operate the application easily. Navigation, labels, forms, dashboard information, error messages, and recommendation messages are reviewed. The interface should allow users to complete common tasks without unnecessary complexity.

11. Defect Identification and Correction

Any defects identified during testing are recorded and analyzed. The development team corrects the affected module and performs the relevant test again. Regression testing is then performed to ensure that fixing one issue does not introduce problems in other parts of the application.

12. Testing Results

The planned test cases are executed for the major modules of PocketSmart AI. The application is checked for correct transaction processing, budget calculations, database operations, dashboard updates, and recommendation generation. Test results are documented and issues found during testing are corrected before the final release.

13. Expected Outcome

The expected outcome of the testing phase is a stable and functional PocketSmart AI application. All major features should operate according to their requirements, financial calculations should be accurate, user data should be handled appropriately, and the AI recommendation feature should provide useful spending-related suggestions.

14. Conclusion

The testing phase ensures that PocketSmart AI is reliable, functional, and ready for the next stage of deployment. Through unit, integration, functional, system, usability, performance, security, and database testing, potential errors can be identified and corrected. Successful testing improves the overall quality and reliability of the final application.